

The scale

5 - Perfect



4 - Strong, only light edits



3 - Approvable, but needs your fixes and alignment



2 - Below the quality bar



"Does not meet the quality bar and is likely a send-back. Consistent 2s are an offboarding signal - repeated work at this level means the fellow is not meeting the bar."

1 - Unusable



One number to confirm

The scoring brief defined 5, 3, 2, and 1 but not 4. I've filled in 4 as "strong, only light edits." Please confirm that's how you want 4 used before this ships, or tell me the intended definition and I'll correct it.

The scale

5 - Perfect



4 - Strong, only light edits



3 - Approvable, but needs your fixes and alignment



2 - Below the quality bar



1 - Unusable



"Completely unusable. The person should most likely be offboarded. Reserve for tasks with no salvageable deliverable."

One number to confirm

The scoring brief defined 5, 3, 2, and 1 but not 4. I've filled in 4 as "strong, only light edits." Please confirm that's how you want 4 used before this ships, or tell me the intended definition and I'll correct it.

The scale

5 - Perfect

4 - Strong, only light edits

3 - Approvable, but needs your fixes and alignment

"The task can be approved, but only after fixes from you, and alignment is needed. You must leave feedback for the attempter so they improve. A 3 means usable work with real gaps."

2 - Below the quality bar

1 - Unusable

One number to confirm

The scoring brief defined 5, 3, 2, and 1 but not 4. I've filled in 4 as "strong, only light edits." Please confirm that's how you want 4 used before this ships, or tell me the intended definition and I'll correct it.

The scale

5 - Perfect



4 - Strong, only light edits



"Approvable with only minor, mechanical edits and little to no alignment needed. Clearly strong work that fell just short of perfect."

3 - Approvable, but needs your fixes and alignment



2 - Below the quality bar



1 - Unusable



One number to confirm

The scoring brief defined 5, 3, 2, and 1 but not 4. I've filled in 4 as "strong, only light edits." Please confirm that's how you want 4 used before this ships, or tell me the intended definition and I'll correct it.

SQS scoring

At the end of every task, you give it a **Single Quality Score (SQS)** from 1 to 5 and a comment explaining why you scored it that way. This is not a formality.

These scores decide who stays on the project

SQS scores are tracked across the organization. High-performing taskers get un-throttled and promoted; low-performing fellows get offboarded. Score accurately and with thought - a careless score either keeps weak work in the pipeline or pushes out a fellow who didn't deserve it.

The scale

5 - Perfect

"A perfect 100%. Prompt, justifications, and rubric are all correct and need no fixes from you. This person is great - flag them as someone to keep and promote."

4 - Strong, only light edits

3 - Approvable, but needs your fixes and alignment

2 - Below the quality bar

1 - Unusable

One number to confirm

The scoring brief defined 5, 3, 2, and 1 but not 4. I've filled in 4 as "strong, only light edits." Please confirm that's how you want 4 used before this ships, or tell me the intended definition and I'll correct it.

Every score needs a justifying comment

Whatever you score, write **why** - tied to the prompt, justifications, and rubric. The comment is what makes the score defensible to the audit layer and useful to the fellow.

Topic	Good	Bad
A defensible score comment	"Scored 3: prompt and failures are solid, but rubric criteria 9 and 12 used process/vibe words and 18 duplicated 17. Fixed all three; alignment feedback left for the attempter."	"Scored 3, seemed about right."

Weak vs. Strong comment set

✓ Good

⚠ Bad

The strong version praises specifics, quotes exact numbers and criterion IDs, says exactly what to change, and confirms that's the complete set for approval.

Practice: Is this comment ready to send?

Decide whether each reviewer comment is good to send, or needs to be rewritten before it goes to the fellow.

0 / 3 decided

"This is wrong, please redo the rubric."

Rewrite first

Send it

"Strong, hard prompt and clean persona. Two fixes to approve: criterion 4 references march_statement.pdf - make it self-contained by stating the value (\$2,939.77); and criterion 18 duplicates 17, so remove 18. That's everything."

Rewrite first

Send it

"I fixed a few things. Looks fine now."

Rewrite first

Send it

Check my work

You can submit now; 3 blank items will count as incorrect.

Previous

Next

Leaving feedback

When you fix or send back a task, your comments are how the fellow learns and how the task gets to an approvable state. Because you own the task through any re-submission, incomplete feedback just buys you another round-trip. Get it all out the first time.

What the ideal comment set does

It tells the fellow what they did well, and gives constructive, specific feedback on everything that needs to change - comprehensively enough that, if they action all of it, you can approve the task. No surprises in round two.

The anatomy of a strong comment set

- 1

Open with what's working

Name the specific things the fellow did well - a strong persona voice, a genuinely hard prompt, a well-targeted justification. This is honest and it tells them what to keep doing.
- 2

Then each issue, in three parts

For every problem: (1) what's wrong, quoted or pointed at specifically; (2) why it matters for the deliverable or the user; (3) exactly what to change. No vague "this is off."
- 3

Close by confirming completeness

State that this is the full set of changes needed to approve. The fellow should be able to action your comments and land an approvable task without a third pass.

The 4 rules

Topic	Good	Bad
Specific	"Quote the exact number, criterion, or sentence. 'Criterion 12 says \$715.95; the correct figure from the files is \$486.91.'"	"Some of the numbers look off."
Actionable	"Say what to do: 'Rewrite criterion 12 to state \$486.91 and re-check the dependent total in 15.'"	"The rubric needs work."
Comprehensive	"Cover everything blocking approval in one pass, across prompt, justifications, and rubric."	"Flag one issue, approve the re-submit, then discover three more."
Constructive	"Neutral, helpful tone that assumes good faith and starts from what worked."	"Harsh or accusatory phrasing that just lists faults."

✓ Good

"Response states that a down payment under 20% triggers PMI."

⚠ Bad

"Response considered the user's PMI exposure."

"Considered" is mind-reading. You can only grade what's written in the response.

Common error indicators - fix criteria that use these

Process words (describe how, not the result): calculates, computes, derives, determines.

Vibe / non-objective words (two graders would disagree): good, clear, helpful, appropriate, reasonable, sufficient, nice, well-structured, properly, adequate, suitable, thorough, accurate, correctly, comprehensive, robust, sound.

Mind-reading words (can't see inside the model): considered, understood, took into account, thought about, analyzed, evaluated, recognized, realized, knew, accounted for.

The fix is almost always the same: rewrite the criterion to state the final value or outcome plainly
- "Response states X."

E. Atomic

Each criterion checks 1 thing, so it can't be half-right.

✓ Good

"(1) Response states the monthly gap is \$1,654.40. (2) Response uses the requested four section headings: What You Actually Have, Monthly Target, Is It Realistic, What To Watch."

⚠ Bad

"Response states the monthly gap is \$1,654.40 and uses the requested four section headings: What You Actually Have, Monthly Target, Is It Realistic, What To Watch."

If two parts could independently pass or fail, split them. Apply with judgment - rubrics are short (20–35 criteria), so don't fragment past the point of usefulness.

Example - Denny (joint statements, provable vs. suspicious duplicates)

"Response correctly classifies duplicates" is not atomic - it bundles five separate calls. Split into: (1) Netflix (\$22.99) classified as provable duplicate, (2) Orangetheory (\$159.00) classified as provable duplicate, (3) Verizon (\$165.00) classified as suspicious-not-proven, (4) Progressive (\$185.00) classified as suspicious-not-proven, (5) Provable-duplicate total stated as \$182.00/month.

C. Self-contained

Every criterion must be gradable from its **own text alone** - no references to anything outside it, especially no file names.

✓ Good

"Response states the Bank of America checking ending balance is \$29,450.67."

⚠ Bad

"Response states the credit card statement value from march_statement.pdf."

A grader with no access to the prompt or the files must be able to check the criterion. Name the final value, not the source.

Common self-contained failures from real tasks

"Response uses the figure from `priya_vasanthan__bofa_checking__202604` ." → name the figure: \$29,450.67.

"Response correctly identifies the duplicates listed in the prompt." → name them: Netflix (\$22.99) and Orangetheory (\$159.00).

"Response matches the SEP contribution limit referenced above." → name it: \$13,535.09 for 2026.

D. Verifiable and objective

A criterion must check an outcome visible in the output - not how the model got there, and not its internal reasoning.

✓ Good

"Response states the monthly card-balance gap is \$1,654.40."

⚠ Bad

"Response calculates the monthly card-balance gap."

"Calculates" describes the model's process. The grader can't see process - only the final value.

✓ Good

"Response states the average monthly credit-card spend is \$5,319.50."

⚠ Bad

"Response gives a clear and helpful budget breakdown."

The good version names the exact values a grader can verify against the response. "Clear" and "helpful" are vibe words - two graders would disagree on whether the bar was met.

Reviewing the rubric

Rubrics run 20–35 criteria, each with a weight from –100 to +100 and a one-line rationale. Check the rubric on 5 things – every example below is a real criterion shape pulled from the golden tasks.

A. Coverage

The rubric must capture the **most important things the prompt asks for**. If a reasonable, correct answer would score well and a bad answer would score poorly, coverage is good. Read the prompt's real asks and confirm each maps to a criterion.

Example – Priya (homebuyer feasibility, 36 months)

The prompt's headline ask is "can I hit a \$165,000 down payment in three years?" Coverage means the rubric has criteria for: (1) the real spendable cash after the \$22,795.95 unpaid tax, (2) the \$2,499.84/month savings target, (3) the explicit 401(k) exclusion, and (4) the four-section deliverable format. Miss any one of those and the rubric does not cover the prompt.

✓ Good

"Response states the monthly savings target to reach \$165,000.00 in 36 months is \$2,499.84."

⚠ Bad

"Response calculates a monthly savings target."

The "Good" version locks in the prompt's headline number. The "Bad" version doesn't cover it – a model that returns \$1,866.62 would still pass.

B. No double jeopardy

Don't penalize the same thing twice

Watch for two criteria checking the same thing, and for a positive/negative pair that mirror each other. If "Response identifies the avalanche method" is +80, there must not also be a "Response fails to identify the avalanche method" at –80. The zero points from a missed positive is already the penalty.

✓ Good

"+60 – Response states the real spendable cash is \$75,005.92 (after the \$22,795.95 tax liability)."

⚠ Bad

"+60 – Response identifies the unpaid tax liability of \$22,795.95. AND –60 – Response fails to subtract the \$22,795.95 tax liability from spendable cash."

Both "Bad" criteria score the same underlying finding. Keep one, and let the zero do the work on a miss.

Answer

Counts - the model reached no valid answer, or no answer at all.

Doesn't count - the model reached the right answer but formatted it differently than you'd prefer.

Instructions

Counts - the model explicitly violated a constraint the prompt stated.

Doesn't count - the model did something the prompt never asked it not to do.

The 2 most common prompt-section problems

1.The prompt doesn't resolve to a correct answer from the files



2.The justification is wrong, so the failure is false



Practice: Is this a real failure?

For each claimed model failure, decide whether it's a meaningful failure worth delivering, or a minor non-failure that should not be the basis of a send-back or justification.

0 / 4 decided

"The model reported March spending of \$8,097.54, but the statements total \$8,282.54 - it dropped a real \$185 charge by wrongly treating it as a duplicate."

Meaningful failure

Not a real failure

"The justification fails the model for 'not reconciling its totals against the account balances,' even though every figure the model reported is correct against the files."

Meaningful failure

Not a real failure

"The model recommended opening a new credit card, which the prompt explicitly forbade."

Meaningful failure

Not a real failure

"The justification dings the model because its executive summary 'could have been phrased more clearly.'"

Meaningful failure

Not a real failure

Check my work

Reviewing the prompt

This is where most send-backs come from, because the prompt can't be edited. Read it as a real person would, then solve it yourself against the environment files before judging anything else.

Always solve the prompt first

You can't tell whether a model failed - or whether a justification is right - until you know the correct answer from the files. Work the prompt end to end before you read the responses.

What a good prompt has

Work through each item.

Checklist

0 of 4 complete

- ☐ **Clearly uses the assigned persona**
The persona's occupation, income, family, and spending show through in the question.
- ☐ **Sounds like a real person**
Natural, first-person phrasing - the kind of question someone actually types, not a templated paragraph.
- ☐ **No LLM-generated text**
Strictly not allowed. Machine-written prompts get sent back.
- ☐ **A real ask that still breaks models**
A genuine personal-finance question that is meaningfully difficult and produces real model failures.

What counts as a meaningful model failure

A meaningful model failure is something the model got **objectively wrong** in a way that changes the answer or violates the prompt. Cosmetic differences, style preferences, and harmless variations don't count.

Hallucination

Counts - the model invented a charge, account, or figure that isn't in the files.

Doesn't count - the model used a slightly awkward synonym for a real figure.

Miscalculation

Counts - wrong total, wrong rate, or a dropped transaction.

Doesn't count - a rounding difference in the last cent that doesn't change the answer.

Files and data

Counts - the model couldn't find the statements, or fabricated data not in the workspace.

Doesn't count - the model opened the files in a different order than you would.

Answer

Counts - the model reached no valid answer, or no answer at all.

Doesn't count - the model reached the right answer but formatted it differently than you'd prefer.

Instructions

Counts - the model explicitly violated a constraint the prompt stated.

Doesn't count - the model did something the prompt never asked it not to do.

The 2 most common prompt-section problems

1. The prompt doesn't resolve to a correct answer from the files



2. The justification is wrong, so the failure is false



Practice: Is this a real failure?

For each claimed model failure, decide whether it's a meaningful failure worth delivering, or a minor non-failure that should not be the basis of a send-back or justification.

0 / 4 decided

"The model reported March spending of \$8,097.54, but the statements total \$8,282.54 - it dropped a real \$185 charge by wrongly treating it as a duplicate."

Meaningful failure

Not a real failure

"The justification fails the model for 'not reconciling its totals against the account balances,' even though every figure the model reported is correct against the files."

Meaningful failure

Not a real failure

"The model recommended opening a new credit card, which the prompt explicitly forbade."

Meaningful failure

Not a real failure

"The justification dings the model because its executive summary 'could have been phrased more clearly.'"

Meaningful failure

Not a real failure

Check my work

Reviewing the prompt

This is where most send-backs come from, because the prompt can't be edited. Read it as a real person would, then solve it yourself against the environment files before judging anything else.

Always solve the prompt first

You can't tell whether a model failed - or whether a justification is right - until you know the correct answer from the files. Work the prompt end to end before you read the responses.

What a good prompt has

Work through each item.

Checklist

0 of 4 complete

- ☐ **Clearly uses the assigned persona**
The persona's occupation, income, family, and spending show through in the question.
- ☐ **Sounds like a real person**
Natural, first-person phrasing - the kind of question someone actually types, not a templated paragraph.
- ☐ **No LLM-generated text**
Strictly not allowed. Machine-written prompts get sent back.
- ☐ **A real ask that still breaks models**
A genuine personal-finance question that is meaningfully difficult and produces real model failures.

What counts as a meaningful model failure

A meaningful model failure is something the model got **objectively wrong** in a way that changes the answer or violates the prompt. Cosmetic differences, style preferences, and harmless variations don't count.

Hallucination

Counts - the model invented a charge, account, or figure that isn't in the files.

Doesn't count - the model used a slightly awkward synonym for a real figure.

Miscalculation

Counts - wrong total, wrong rate, or a dropped transaction.

Doesn't count - a rounding difference in the last cent that doesn't change the answer.

Files and data

Counts - the model couldn't find the statements, or fabricated data not in the workspace.

Doesn't count - the model opened the files in a different order than you would.

Workflow and send-backs

Your default is to fix and improve the task, not reject it. Most issues are minor and fixable in place. Send-backs are reserved for a small set of real failures.

The one-line rule

Fix what you can. Send back only a broken prompt, a task with no real model failure, or a rubric too poor to salvage. Everything else is a minor edit you make yourself.

When to fix vs. Send back

Situation	Action	Avoid
Rubric wording is sloppy	Fix it - tighten the criterion, correct the weight, move on.	Sending the whole task back over an editable rubric line.
A justification is vague but the failure is real	Fix it - sharpen the justification with the exact numbers.	Rejecting a task that has a genuine model failure.
The prompt doesn't resolve to a correct answer	Send it back - prompts can't be edited.	Trying to patch the prompt yourself, or approving it anyway.
No model actually failed	Send it back - there's nothing to deliver.	Manufacturing a failure to clear the difficulty bar.

Hard rule - prompt problems are always a send-back

Prompts cannot be edited by reviewers. If the prompt is broken - wrong, ambiguous, off-persona, or it doesn't resolve against the files - the only path is to send it back to the fellow with feedback. You can never fix a prompt in place.

Your review order

- 1 The prompt**
Persona, natural phrasing, no LLM-written text, and - most important - it resolves to a single correct answer from the environment files.
- 2 The model responses and justifications**
Confirm the failures are meaningful, not minor, and that each justification's numbers are actually correct against the files.
- 3 The rubric**
Coverage, no double jeopardy, self-contained, verifiable and objective, atomic.

Live sessions

- Reviewer office hours with Malik and Mo, Tuesdays, 4:00 PM PT - [Join](#)
- Reviewer office hours with Malik and Mo, Fridays, 11:00 AM PT - [Join](#)

Getting your first 2 reviews signed off

Before you get unthrottled, your first 2 reviews must be signed off by a team member. After you submit each of your first 2 reviews, post a message in the reviewer Slack channel, tagging the project team, with the following:

Task ID: ...
Decision: Approve/Reject
SQS: (Score out of 5)
Notes:

 Copy

Use **Notes** for a brief heads-up on anything the team should be aware of when reviewing your review - edge cases you weren't sure about, judgment calls you made, or context that explains your decision.

A project specialist will review your decision and reasoning, and reply in thread to let you know whether they agree with your review or if anything needs to change. Once both reviews are signed off and meet the quality bar, you'll be unthrottled and can review at full capacity.

Owning a task

When you claim a task, you own it end-to-end. You must see every task all the way through to the end - no exceptions.

Claiming a task is a commitment, not a placeholder

The moment you claim a task, it is yours until it is fully resolved - approved or finally rejected after every re-submission. Walking away from a claimed task, letting it sit overnight, or expecting someone else to finish it is not acceptable. Reviewers who don't see their tasks through get pulled from the project.

- **Finish what you start - same day, every time.** Every task you claim must be completed the same day you claim it. Do not leave a claimed task open overnight. If you can't finish it today, don't claim it.
- **See every send-back through to resolution.** When a fellow re-submits, you are the 1 who reviews the re-submission - and the 1 after that, and the 1 after that, until the task is approved or finally closed. Sending a task back is not the end of your job; it is the middle of it.
- **Never abandon a claim.** A claimed task is not a maybe. Do not drop it for someone else to pick up, do not unclaim it because it got harder than you expected, and do not leave it half-reviewed. Own it to resolution.

Previous

Next

Your role as a reviewer

Reviewers are the quality gate on Project Lighthouse. Every task that passes through you is delivered to the client. 3 pieces make up that delivery:

- **The prompt** - the consumer-finance question the fellow wrote.
- **The model failure justifications** - the written case for why specific models failed.
- **The rubric** - the weighted criteria used to score the model responses.

The **role introduction** attached to each task is context for you to understand the scenario - you review and edit it, it is not itself a deliverable.

You are the approval - and approval releases payment

Lighthouse pays per approved task, and you are the approval. When you approve, a fellow gets paid for that task. A loose approval pays out work that should not have passed; an unfair rejection costs a fellow work that was actually fine. Keep that in mind on every review.

Why being a reviewer is worth it

- **Hourly pay** - Reviewers are paid hourly, (based on the task time limit)
- **A higher task limit** - Reviewing lifts your ceiling on the project.
- **Direct influence on the project** - Your feedback shapes how the project runs and what "good" means.
- **Priority for future projects** - Your work is tracked on the backend, so strong reviewers get prioritized for other projects as they open up.

Getting started - The onboarding gate

- 1 Attend the onboarding session**
Required before you review anything.
- 2 Start capped at 1-2 reviews**
New reviewers are throttled while a specialist checks their early work.
- 3 Get signed off to unthrottle**
After your first one or two reviews, post in Slack and get sign-off from one of the project specialists. If your work meets the quality bar, they unthrottle you and you can continue reviewing at full capacity.

Your reviews are audited

A downstream audit layer spot-checks reviewer decisions. Over-rejections and approvals of tasks that don't meet the quality bar both show up. Reviewers who consistently miss in either direction get coached, and patterns that don't improve may affect your spot on the project. Your approvals carry weight - treat them that way.

Live sessions

- Reviewer office hours with Malik and Mo, Tuesdays, 4:00 PM PT - [Join](#)
- Reviewer office hours with Malik and Mo, Fridays, 11:00 AM PT - [Join](#)